

UT-CORE CubeSat Flight Software

Modular, Fault-Tolerant Embedded Systems for Space Applications

Role: Software Team (Embedded Firmware) | **Team:** 10 engineers (6 ME, 3 EE, 1 CE)

Open-Source Project | Utah Tech University

<https://github.com/UT-Core-CubeSat>

Overview

The UT-CORE CubeSat project is a university-scale satellite program at Utah Tech, sponsored by NASA, the University Nanosatellite Program (UNP), and the Utah Water Conservancy District for an eventual Earth-imaging mission. The core challenge was designing a modular flight software framework that future teams can reuse by simply adapting a new payload to existing, proven subsystems — eliminating the need to rebuild the entire satellite stack each year. The team of 10 engineers was organized into software, electrical, controls, and structures sub-teams. I worked on the software team, responsible for all flight firmware running across seven custom PCBs in a distributed, fault-tolerant architecture. The entire project is open source.

My Contributions

- Developed flight firmware in C for a distributed multi-board CubeSat architecture, with each of the seven custom boards running Zephyr RTOS on STM32U5 microcontrollers
- Implemented multi-threaded application code on single-core MCUs using Zephyr's kernel primitives (`k_thread_create`, priority scheduling, semaphores) to manage time-critical tasks including CAN message processing, watchdog servicing, GNSS polling, and telemetry assembly
- Built the CAN bus communication layer enabling inter-board messaging at 500 kbps, leveraging CAN's differential signaling for noise immunity in the space environment and its arbitration-based protocol for deterministic, fault-tolerant communication
- Developed drivers for on-board peripherals (temperature sensors, eMMC, FRAM) over SPI, I2C, and UART interfaces local to each subsystem board
- Implemented triple modular redundancy (TMR) logic across three radiation-tolerant FRAM chips per board, with voting and error-correction routines to ensure data integrity against single-event upsets
- Wrote firmware for dual-MCU redundancy on each board, including runtime detection of the active chip and the ability to reprogram the secondary MCU if the primary experiences a fault
- Designed and implemented the CDH (Command & Data Handling) logic flow encompassing setup mode, standard operation mode, error correction mode, and mission mode with autonomous state transitions based on system health telemetry
- Debugged complex hardware–software integration issues using oscilloscopes, logic analyzers, and JTAG/SWD debugging tools across multiple custom boards simultaneously

- Assisted electrical engineers with PCB design decisions, ensuring firmware requirements (pin mapping, bus routing, peripheral placement) were reflected in the board layouts
- Followed disciplined software engineering practices including version control with Git, modular code architecture, and documented APIs to support long-term maintainability across future student teams
- Gained hands-on systems engineering experience coordinating across software, electrical, controls, and structures teams to manage integration of a complex multi-disciplinary aerospace system

Technical Details

Hardware: 7 custom PCBs, STM32U5 MCUs (dual per board for redundancy), radiation-tolerant FRAM (3 per board, TMR), eMMC storage, temperature sensors, CAN transceivers

Communication: CAN bus at 500 kbps (inter-board), SPI / I2C / UART (intra-board peripherals)

Software: Embedded C with Zephyr RTOS abstractions, multi-threaded architecture on single-core MCUs, priority-based preemptive scheduling. Software layers include: Hardware Abstraction Layer (HAL), Driver Layer (SPI, CAN, I2C, FRAM), Middleware (messaging, fault detection), and Application Layer (ADCS, CDH, EPS logic)

Toolchain: Zephyr RTOS + West build system, STM32CubeProgrammer, JTAG/SWD via ST-Link, MATLAB (ADCS modeling and validation), oscilloscope + logic analyzer for bus debugging, Git for version control

Key Design Decisions

Zephyr RTOS: Chosen for its hardware abstraction, portability across STM32 targets, and mature threading and scheduling primitives. Enables the same application code to run on any subsystem board with minimal modification, which is critical for the project's modularity goal.

CAN Bus: Selected over alternatives like UART or SPI for inter-board communication because of its differential signaling (noise-resilient in the space radiation environment), built-in arbitration (no master required), error detection via CRC, and proven heritage in automotive and aerospace systems.

Dual-MCU Redundancy: Each board carries two STM32U5 chips. Firmware detects which chip is currently active at boot and can reprogram the cold spare if a fault is detected, providing single-fault tolerance at the processor level.

Triple Modular Redundancy (FRAM): Three radiation-tolerant FRAM chips per board store critical data. A majority-vote scheme detects and corrects single-chip bit flips caused by radiation-induced single-event upsets, ensuring persistent data integrity without external error-correction hardware.

Project Figures

```
int main(void)
{
    LOG_INF("UT-CORE CDH Booting...");

    leds_init();

    /* Priority 0 - watchdog must be first and highest */
    k_thread_create(&watchdog_thread_data, watchdog_stack, STACK_SIZE,
        watchdog_thread, NULL, NULL, NULL,
        PRIO_WATCHDOG, 0, K_NO_WAIT);

    /* Priority 1 - CAN RX must be near the top to drain the hardware FIFO */
    k_thread_create(&can_thread_data, can_stack, STACK_SIZE,
        can_rx_thread, NULL, NULL, NULL,
        PRIO_CAN_RX, 0, K_NO_WAIT);

    /* Priority 2 - CAN frame decode and dispatch */
    k_thread_create(&can_proc_thread_data, can_proc_stack, STACK_SIZE,
        can_process_thread, NULL, NULL, NULL,
        PRIO_CAN_PROC, 0, K_NO_WAIT);

    /* Priority 3 - heartbeat + future mission scheduling */
    k_thread_create(&sched_thread_data, sched_stack, STACK_SIZE,
        scheduler_thread, NULL, NULL, NULL,
        PRIO_SCHED, 0, K_NO_WAIT);

    /* Priority 4 - GNSS position requests and pass-window checks */
    k_thread_create(&gnss_thread_data, gnss_stack, STACK_SIZE,
        gnss_thread, NULL, NULL, NULL,
        PRIO_GNSS, 0, K_NO_WAIT);

    /* Priority 5 - temperature reads and node liveness checks */
    k_thread_create(&soh_thread_data, soh_stack, STACK_SIZE,
        soh_thread, NULL, NULL, NULL,
        PRIO_SOH, 0, K_NO_WAIT);

    /* Priority 6 - lowest; telemetry assembly and COMMS downlink handoff */
    k_thread_create(&telemetry_thread_data, telemetry_stack, STACK_SIZE,
        telemetry_thread, NULL, NULL, NULL,
        PRIO_TELEMETRY, 0, K_NO_WAIT);

    setup_mode_init();

    while (1) {
        k_sleep(K_FOREVER);
    }
}
```

Figure 1: CDH main thread architecture showing priority-based task creation in Zephyr RTOS — from watchdog (priority 0) through CAN RX, CAN processing, scheduling, GNSS, state-of-health, down to telemetry assembly (priority 6).

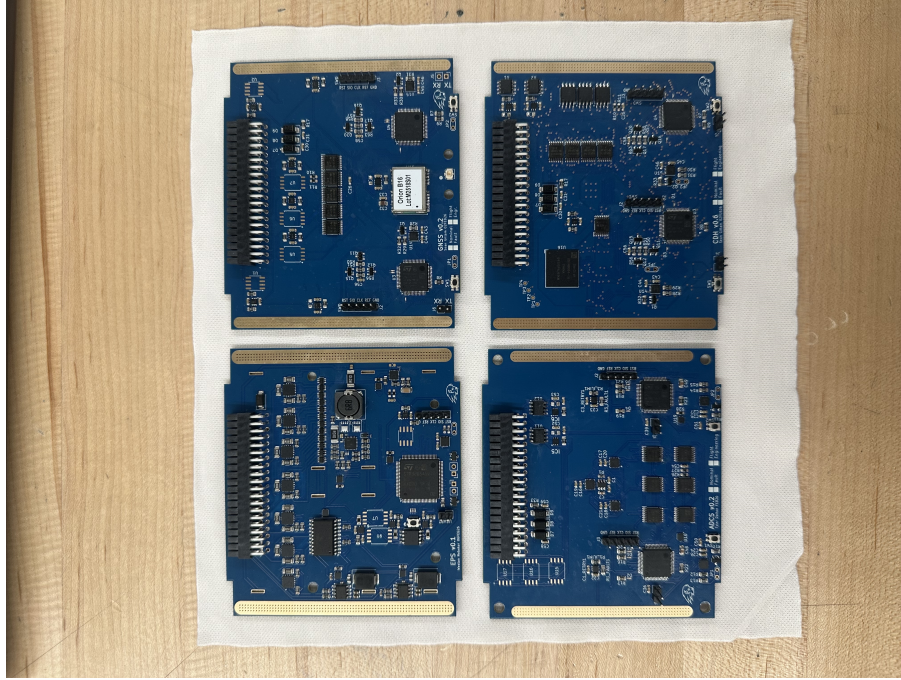


Figure 2: Four custom subsystem boards running verified flight firmware: EPS v0.1, GNSS v0.2, ADCS v0.2, and CDH v0.6. Each board features dual STM32U5 MCUs, triple FRAM, CAN transceivers, and gold edge connectors for the PC/104 stack.



Figure 3: The UT-CORE CubeSat team at the Science, Engineering & Technology Building, Utah Tech University.



Figure 4: Fully assembled CubeSat structure with machined aluminum chassis, board slot rails, and external D-sub connector for ground-station interfacing.